Concurrency Control & Recovery

Study notes with diagrams: locking, deadlocks, atomicity, undo/redo and remote backup

#	Topic	What you will learn
1	Phase locking (lock-based protocols)	Why locks are needed, S/X modes, compatibility
2	Two-phase locking	Growing/shrinking phases, lock point, strict and rigorous 2PL
3	Lock conversion	Upgrade and downgrade rules
4	Deadlocks	Wait-for graph, wait-die, wound-wait, detection and recovery
5	Data access, recovery and atomicity	Buffer and disk, transaction states, write-ahead log
6	Undo and redo operations	Log records, checkpoint, two-pass recovery example
7	Remote backup systems	Log shipping, failover, 1-safe and 2-safe

1. Phase Locking (Lock-Based Protocols)

When many transactions run at the same time, uncontrolled interleaving can produce wrong results: **lost update**, **dirty read** (reading uncommitted data) and **unrepeatable read**. A **lock** is a variable attached to a data item that tells other transactions what they may do with it. A transaction must obtain a lock before accessing an item, and the lock manager grants it only if it is compatible with locks already held.

Lock modes

- **Shared (S)**: the holder may only **read** the item. Many transactions can hold S on the same item.
- **Exclusive (X)**: the holder may **read and write**. Only one transaction can hold X, and no S lock may coexist with it.

Lock compatibility matrix

Requested \ Held	S	X
S	Granted	Wait
X	Wait	Wait

Basic rules: lock-S(Q) before read(Q); lock-X(Q) before write(Q); unlock(Q) when finished. A blocked request waits in a queue until the conflicting lock is released.

> **Key point:** Locking alone does not guarantee serializability. If a transaction unlocks an item too early and then locks another, other transactions can slip in between and the schedule may become non-serializable. We therefore need a **rule about when to lock and unlock**, which is the two-phase protocol.

2. Two-Phase Locking (2PL)

The two-phase locking protocol requires every transaction to issue lock and unlock requests in **two phases**:

- **Growing phase**: the transaction may obtain locks but may not release any.
- **Shrinking phase**: the transaction may release locks but may not obtain new ones. Once the first unlock happens, it is in this phase for good.

The moment the transaction holds its maximum number of locks (end of growing phase) is the **lock point**. Ordering transactions by their lock points gives an equivalent serial order, so **2PL guarantees conflict-serializable schedules**.

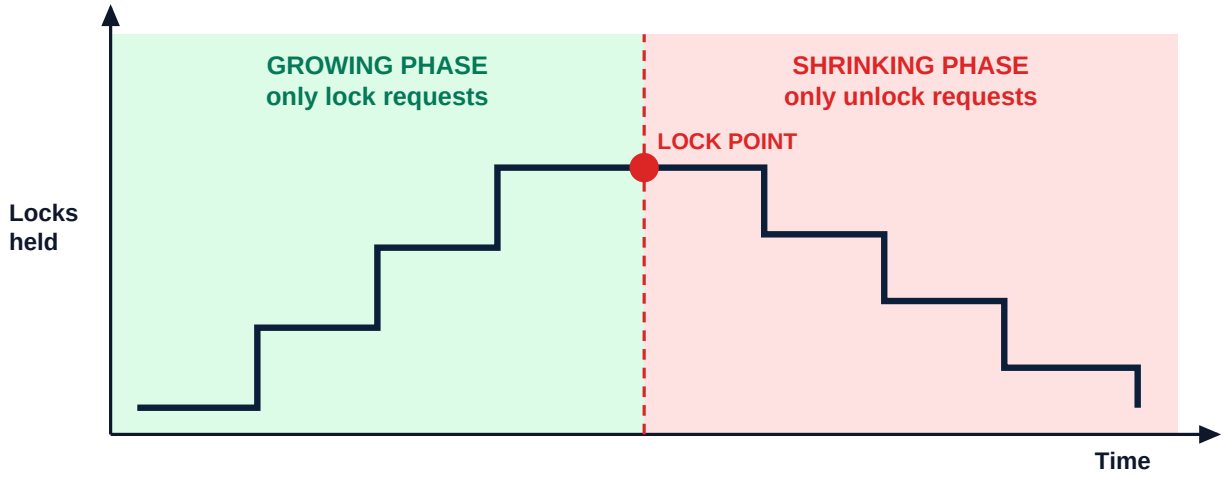

Figure 1: Number of locks held over time under two-phase locking

Variants of 2PL

Variant	Rule	Effect
Basic 2PL	Growing then shrinking phase	Serializable, but cascading rollback and deadlock are possible
Strict 2PL	Holds all **X** locks until commit or abort	Avoids cascading rollbacks (no dirty reads)
Rigorous 2PL	Holds **all** locks until commit or abort	Serial order equals commit order; simplest to implement
Conservative 2PL	Acquires all locks **before** the transaction starts	Deadlock-free, but must know the lock set in advance

Limitations

- 2PL does **not** prevent deadlock (except conservative 2PL).
- Basic 2PL does not prevent cascading rollback, so real systems use strict or rigorous 2PL.
- Some serializable schedules cannot be produced by 2PL, so concurrency is not maximal.

3. Lock Conversion

Sometimes a transaction first needs only to read an item and later decides to update it. Instead of taking X immediately (which blocks all readers), it can start with S and **convert** the lock when needed.

- **Upgrade**: S to X. Allowed only in the growing phase, because it is like acquiring a new, stronger lock.
- **Downgrade**: X to S. Allowed only in the shrinking phase, because it is like releasing part of a lock.

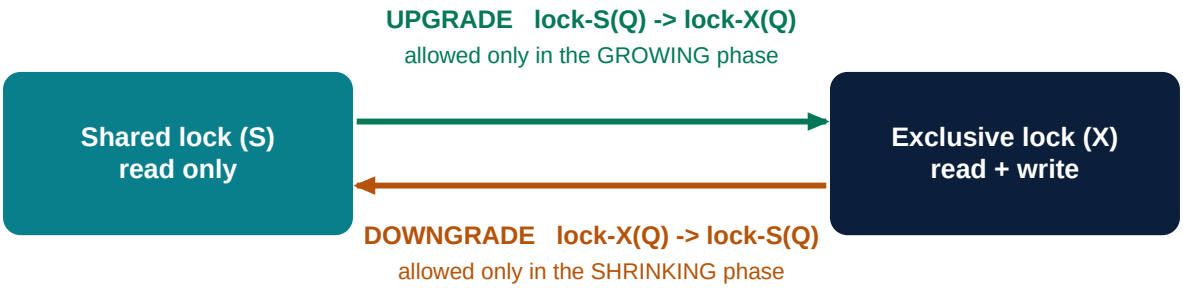

Figure 2: Lock conversion between shared and exclusive modes

Example

```
T1: lock-S(A); read(A);
lock-S(B); read(B);
lock-upgrade(A); <- S to X (growing phase)
write(A);
unlock(A); unlock(B);
```

Why it helps: readers can share the item for longer, so **concurrency improves**. Watch out: if two transactions both hold S on Q and both try to upgrade, each waits for the other to release, which causes a **conversion deadlock**. Using 2PL with lock conversion still guarantees serializability.

> **Remember:** upgrade = growing phase only; downgrade = shrinking phase only. This is what keeps conversion consistent with two-phase locking.

4. Deadlocks

A **deadlock** occurs when a set of transactions each wait for a lock held by another transaction in the set, so none of them can ever proceed. Example: T1 holds X(A) and requests B, while T2 holds X(B) and requests A.

Four necessary conditions

- **Mutual exclusion**: a resource is held in exclusive mode.
- **Hold and wait**: a transaction holds one lock while waiting for another.
- **No preemption**: locks cannot be forcibly taken away.
- **Circular wait**: a closed chain of waiting transactions exists.

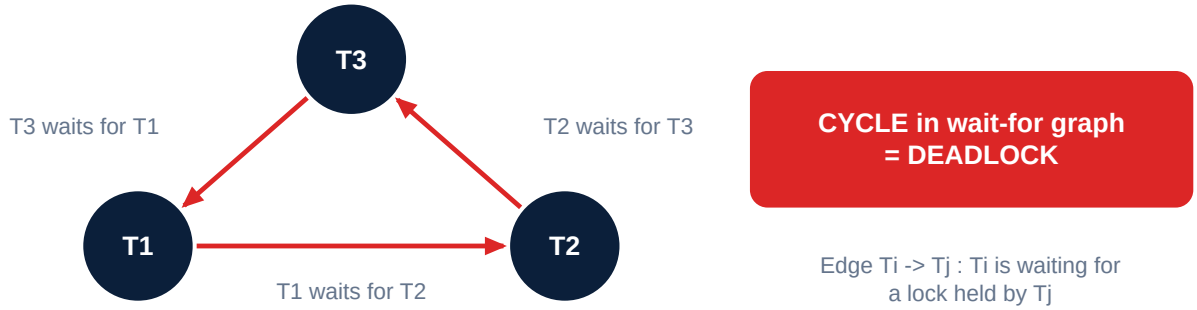

Figure 3: Wait-for graph with a cycle (deadlock)

Deadlock prevention

- **Ordering**: lock items in a fixed global order, or use conservative 2PL (lock everything first).
- **Timestamp schemes** using transaction age (older = smaller timestamp), compared below.
- **Lock timeout**: abort a transaction that waits longer than a limit (simple, but may abort needlessly).

Situation	Wait-die (non-preemptive)	Wound-wait (preemptive)
Older requests lock held by younger	Older **waits**	Older **wounds** younger (younger rolls back)
Younger requests lock held by older	Younger **dies** (rolls back)	Younger **waits**

Both schemes restart the victim with its **original timestamp**, so it eventually becomes the oldest and cannot starve.

Deadlock detection and recovery

- The system maintains a **wait-for graph** and checks it periodically; a **cycle means deadlock**.
- **Victim selection**: choose the transaction with minimum cost (work done, locks held, number of times already rolled back) to avoid starvation.
- **Rollback**: total (abort and restart) or partial (roll back only far enough to release the needed lock).

5. Data Access, Recovery and Atomicity

Data is stored in disk **blocks**. A transaction moves data between disk and a main-memory **buffer** using **input(B)** and **output(B)**, and between the buffer and its own workspace using **read(X)** and **write(X)**. A write changes only the buffer first; the block reaches disk later, so a crash can leave the database in a partly updated state.

Storage types and failures

Storage	Survives crash?	Example
Volatile	No	Main memory, cache, buffer
Non-volatile	Yes (system crash), not disk failure	Disk, SSD
Stable (idealized)	Yes, survives all failures	Replicated disks, remote copies

- **Transaction failure**: logical error or deadlock victim.
- **System crash**: power loss or software fault; volatile memory is lost.
- **Disk failure**: media is damaged; need backups or remote copies.

Atomicity means **all or nothing**: either every update of a transaction is reflected in the database or none is. The recovery manager enforces it (together with durability) using a log.

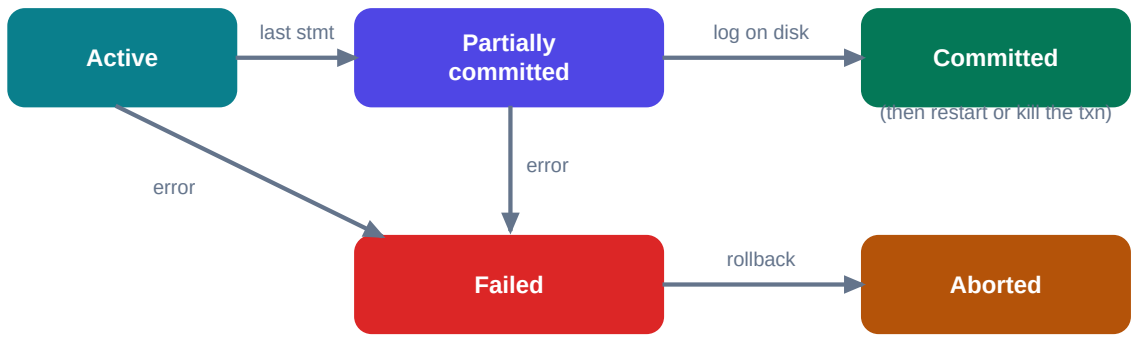

Figure 4: Transaction state diagram

Log-based recovery

- The **log** is a sequence of records kept on stable storage: <Ti start>, <Ti, X, old, new>, <Ti commit>, <Ti abort>.
- **Write-ahead logging (WAL)**: a log record must reach stable storage **before** the data block it describes, and all of a transaction's log records (including commit) must be on disk before it is declared committed.
- **Deferred modification**: DB written only after commit (needs redo only). **Immediate modification**: DB may be written before commit (needs both undo and redo).
- **Shadow paging** is an alternative: keep a shadow page table and switch to the new table atomically on commit; no log needed, but data becomes fragmented.

6. Undo and Redo Operations

Each update log record stores both the **old value** and the **new value**, which makes two inverse operations possible:

- **undo(Ti)**: scan the log **backward**; for every <Ti, X, V1, V2> restore X = V1 and write a compensation record. At <Ti start> write <Ti abort>.
- **redo(Ti)**: scan the log **forward**; for every <Ti, X, V1, V2> set X = V2. Redo is **idempotent**: doing it twice gives the same result.

Recovery after a crash

- **Checkpoint**: periodically force log and dirty buffers to disk and write <checkpoint>, so recovery need not scan the whole log.
- **Redo phase** (forward from the checkpoint): repeat history; every transaction seen without a commit or abort goes to the **undo-list**.
- **Undo phase** (backward): undo every transaction in the undo-list.
- Rule of thumb: **committed = redo, uncommitted = undo**.

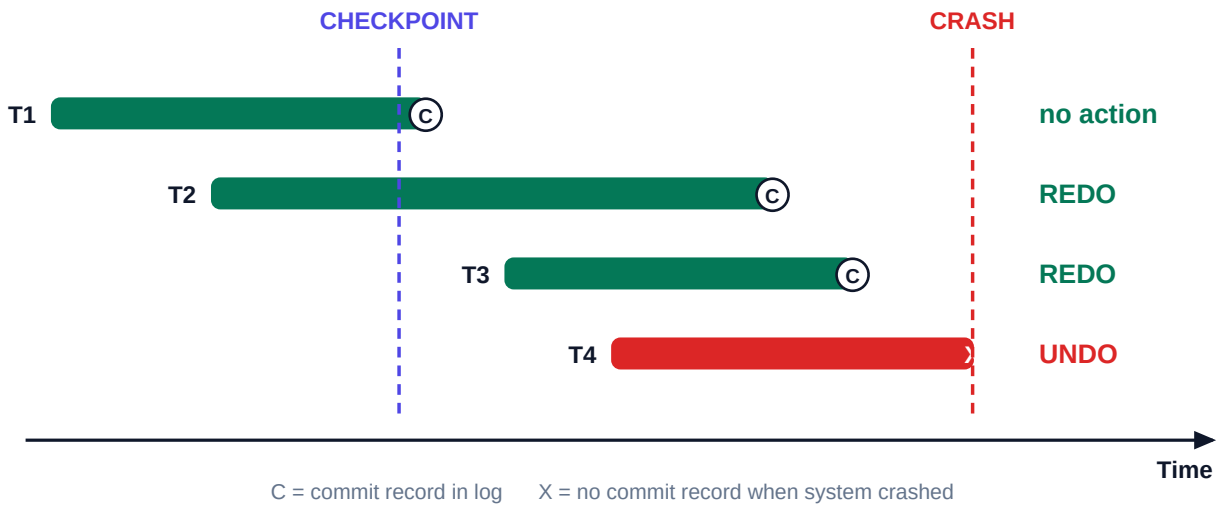

Figure 5: Which transactions are redone or undone after a crash

Worked example

```
1 <T1 start>
2 <T1, A, 100, 150>
3 <T1 commit>
4 <checkpoint>
5 <T2 start>
6 <T2, B, 200, 250>
7 <T2 commit>
8 <T3 start>
9 <T3, C, 300, 350> ---- CRASH ----
```

Transaction	Status in log	Action	Result
T1	Committed before checkpoint	None	A = 150 (already on disk)
T2	Committed after checkpoint	**Redo**	B = 250
T3	Started, no commit	**Undo**	C = 300 (old value restored), then <T3 abort>

7. Remote Backup Systems

A single site can be lost completely (fire, flood, power or network outage). A **remote backup system** keeps a copy of the database at a geographically separate site so service can continue after a disaster. It is also called a hot-standby or replicated system.

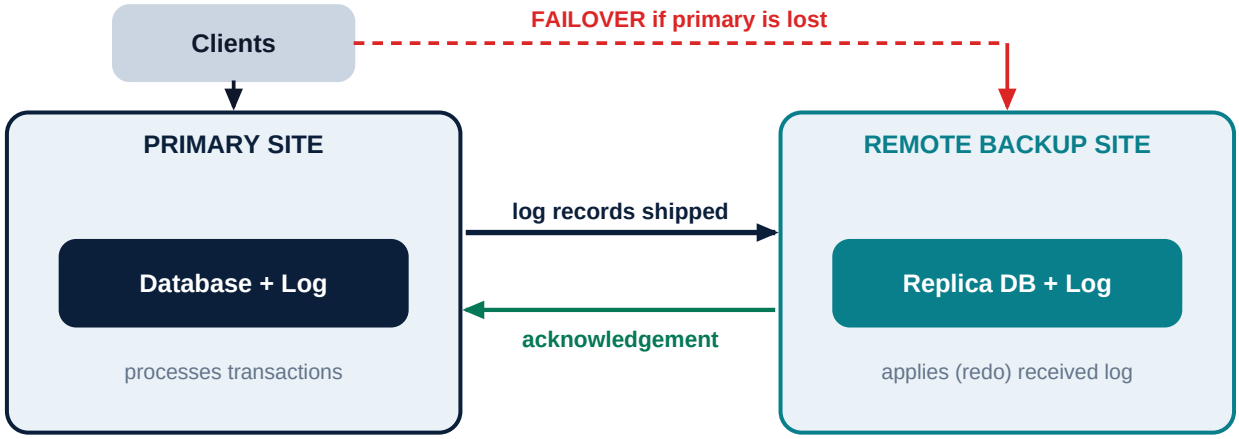

Figure 6: Primary site shipping its log to a remote backup site

How it works

- The primary processes all transactions and **ships log records** to the backup (cheaper than shipping data pages).
- The backup **applies (redoes)** the received log continuously, so it stays almost up to date.
- **Failure detection**: heartbeat messages; a missing heartbeat from the primary may mean a crash or just a network fault, so use several communication links to avoid a false takeover.
- **Transfer of control**: the backup finishes redo, rolls back incomplete transactions (undo) and becomes the new primary. The old primary later catches up from the new primary's log.

Commit durability levels

Level	Transaction commits when...	Trade-off
1-safe	Commit log record reaches stable storage at the **primary**	Fastest; a few committed transactions may be lost if primary fails before shipping
2-safe	Log reaches **both** sites (if backup is down, behaves as 1-safe)	No loss when both up; slower commit
2-very-safe	Log reaches **both** sites, always	No loss ever, but unavailable if either site is down

> **Design goals: High availability** (short time to take over) and **no lost commits**. Faster takeover needs a hot standby; stronger durability costs commit latency.

Quick revision

Topic	One-line summary
Phase locking	Locks (S/X) control access, but need a protocol to ensure serializability
Two-phase locking	Grow then shrink; lock point gives serial order; strict/rigorous 2PL avoid cascades
Lock conversion	Upgrade S to X while growing; downgrade X to S while shrinking
Deadlocks	Cycle in wait-for graph; prevent (wait-die, wound-wait) or detect and abort a victim
Atomicity and recovery	All-or-nothing, enforced with write-ahead logging on stable storage
Undo and redo	Redo committed work forward, undo uncommitted work backward, from last checkpoint
Remote backup	Ship log to a standby site; fail over; choose 1-safe, 2-safe or 2-very-safe